

Freight Rate Prediction

Technical Report — Machine Learning Engineer Assessment

Siddharth Muneshwar

Repository: `github.com/SiddharthMuneshwar26/freight-rate-prediction`

Deliverables: `validation_predictions.csv` (12,000 rows) · `december-chart-inputs.csv` (31 rows) · `scorer_results/candidate_december.png`

1. Executive summary

I trained a leakage-safe freight-rate regressor on 48,000 labelled loads (Jan–Oct 2025), predicted all 12,000 future loads (Nov–Dec 2025), and produced the fixed December scenario series required by the supplied scorer. Every number in this report is generated by `run_pipeline.py` and stored in machine-readable artifacts under `artifacts/`; nothing is hand-transcribed.

	Selected model	Feature set	Chronological holdout MAE
Main validation model	<code>hgb_compact</code> (HistGradientBoostingRegressor)	<code>basic_plus_geographic</code>	\$129.51
December scenario model	<code>december_hgb__december_basic</code> (HistGradientBoostingRegressor)	<code>december_basic</code>	\$108.78

Against the strongest naive rule available (median rate-per-mile by equipment, scaled by distance), the selected model reduces MAE by 43%, from \$229.10 to \$129.51. Against a global median it reduces MAE by 89%.

Four findings drove the design and are worth reading before the numbers:

- The target is near-deterministic, with 1.4% deliberate contamination.** After normalising rate-per-mile by its local median, 98.6% of loads sit within $\pm 12\%$ of that median ($\sigma = 5.1\%$). The remainder splits cleanly into 340 loads inflated $2.1\times$ – $5.3\times$ and 337 loads deflated to $0.17\times$ – $0.47\times$. On the holdout, those contaminated rows are 1.5% of the data and carry **43.7% of all absolute error**.
- This makes the metric choice decisive.** Optimising squared error would let 1.5% of rows dictate the model. I train and select on absolute error throughout, on a rate-per-mile target. On uncontaminated rows the model achieves **\$74 MAE**, and 98.4% of predictions land within $\pm 10\%$ of the actual rate.
- Rate levels drift upward across 2025 and decouple from `market_index` after July.** The selected feature set carries no calendar signal, so the model reproduces the *shape* of the rate surface accurately but inherits a level offset of **–\$105 per load** on the holdout. I quantify this in §8 rather than leaving it implicit, and show a validated correction worth ~ 16 – 18% of MAE.
- The results reproduce exactly across machines.** A clean re-run on Linux (pandas 3.0.2 / scikit-learn 1.8.0) reproduced the committed 12,000-row prediction file to within 9×10^{-13} of the original Windows run.

2. Task and constraints

The brief supplies `train-test.csv` (labelled), `validation.csv` (unlabelled, later in time), a fixed 31-row December scenario, and an unmodifiable `score.py`. The candidate decides how to split and validate, then submits predictions for every validation load.

Two properties of the setup shaped everything downstream:

- The final validation set is strictly later in time than all labelled data.** Development ends 2025-10-31; validation begins 2025-11-01. This is a forecasting problem, not a random-holdout problem, and any validation scheme that shuffles rows would overstate performance.
- The final validation labels are never visible.** Nothing in this report is a test score. Every reported metric is a *model-selection* estimate measured on a chronological holdout carved out of the labelled data, and I label it as such throughout.

3. Data audit

3.1 Shape and schema

	Rows	Columns	Date range
Development (<code>train-test.csv</code>)	48,000	14	2025-01-01 to 2025-10-31
Final validation (<code>validation.csv</code>)	12,000	13	2025-11-01 to 2025-12-31
December scenario	31	7	2025-12-01 to 2025-12-31

The target column is not named in the brief, so the pipeline *infers* it: it is the single column present in development and absent from validation (`posted_rate`), and the pipeline fails loudly if that set is not of size one. There are no duplicate rows, no duplicate `load_id` values, and no unparseable dates in either file.

`posted_rate` is right-skewed (skew 1.90): median \$2,030.76, mean \$2,373.98, range \$57.22 – \$25,533.00, with the 5th–95th percentiles spanning \$599.74 – \$4,953.77.

Equipment splits into Dry Van (27,202), Reefer (12,045) and Flatbed (8,753). There are 64 distinct cities and 4,014 distinct routes in development.

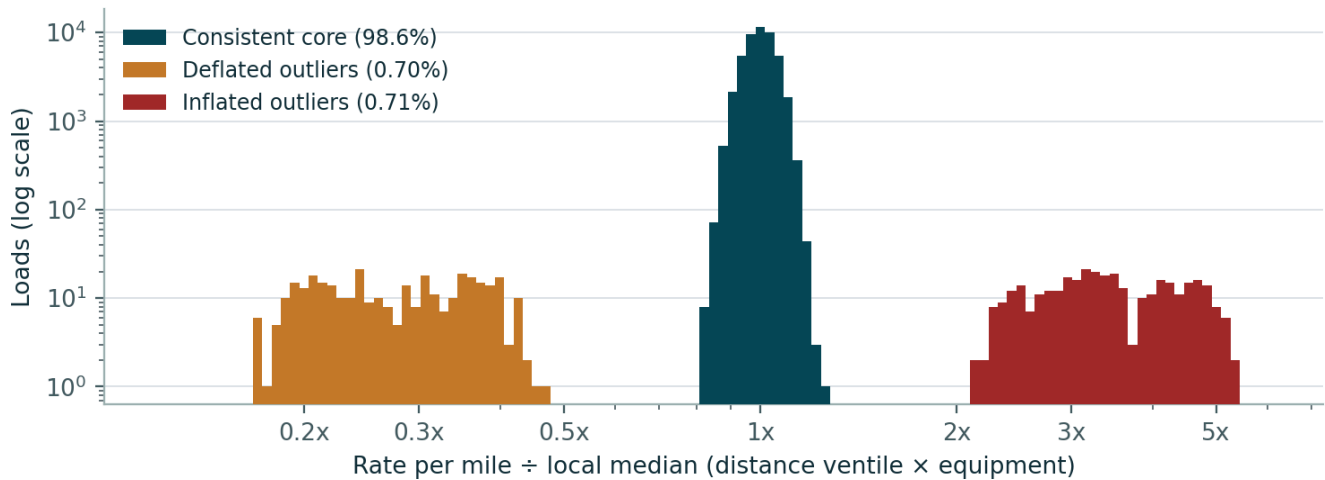
3.2 Data-quality issues found and how each was handled

Issue	Extent	Treatment	Rationale
Missing <code>weight</code>	300 dev (0.63%), 165 val (1.38%)	Left missing; <code>weight_missing</code> flag added	Both model families handle NaN natively; imputing would erase the fact that the value was absent, which is itself predictive
Missing <code>market_index</code>	374 dev (0.78%), 249 val (2.08%)	Left missing; <code>market_index_missing</code> flag added	As above
Impossible negative <code>weight</code>	292 dev, 145 val	Converted to missing; <code>weight_sign_corrected</code> flag retained	A –36,485 lb load is a data error, not a light load; keeping the sign would corrupt any weight-based split
Inconsistent category text	Present across <code>pickup</code> / <code>delivery</code> / <code>equipment</code>	Whitespace collapsed, title-cased, empty to <code>Unknown</code>	Prevents <code>"dry van"</code> and <code>"Dry Van"</code> becoming separate categories
Extreme target values	1.4% of rows (see §3.3)	Retained , not clipped or removed	These are the phenomenon, not noise to be swept away; the absolute-error objective bounds their influence without discarding real loads
Coordinates appear synthetic	<code>distance</code> vs great-circle correlation 0.99953, median ratio 1.182	Supplied <code>distance</code> treated as authoritative	Coordinates are internally consistent but obfuscated; the constant 1.18 inflation over great-circle distance suggests a generated road-distance factor
Leading-zero-safe IDs	—	<code>load_id</code> read as text everywhere	Guarantees the submitted IDs match the template byte-for-byte

3.3 The target has a three-mode structure

Dividing each load's rate-per-mile by the median rate-per-mile of its own distance-ventile × equipment cell removes the two dominant drivers and exposes what is left:

Posted rates are near-deterministic apart from 1.4% multiplicative outliers



- **98.6% of loads** fall in a tight core: $\sigma = 5.1\%$, 1st-99th percentile $0.887\times - 1.119\times$.
- **340 loads (0.71%)** are inflated by $2.10\times - 5.28\times$ (median $3.37\times$).
- **337 loads (0.70%)** are deflated to $0.167\times - 0.471\times$ (median $0.28\times$).

The contamination is uniform in time (0.5%-0.8% per month) and uniform across equipment types, and it is uncorrelated with the negative-weight rows or the missing- `market_index` rows — so it is not a downstream symptom of the other data-quality defects, and there is no exploitable rule for identifying a contaminated row from its features.

This is the single most consequential fact about the dataset. It means the honest description of model quality is two-part: a tight fit on the 98.6%, and a floor on aggregate error set by the 1.4% that no feature set can anticipate.

3.4 Development-to-validation shift

Feature	Development mean	Validation mean	Standardised difference
<code>market_index</code>	1.0834	0.9269	-1.205
<code>quote_signal</code>	2.0625	2.0513	-0.043
<code>weight</code>	31,417	31,283	-0.017
<code>distance</code>	1,135.9	1,141.8	+0.008

`market_index` is the only materially shifted feature. The shift is against the *full* Jan-Oct average, however — validation's 0.927 sits close to the Sep-Oct level of 0.89-0.96, so the validation period is not an unprecedented regime, it is a continuation of the most recent one. §8 returns to why that distinction matters.

Categorically, validation introduces **8 cities never seen in development** — Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk and San Diego — affecting 725 rows as pickup and 722 as delivery. 1,461 validation rows sit on routes absent from development, and a further 2,692 on routes seen fewer than 10 times. Equipment has no unseen categories.

4. Validation strategy and data split

This is the part of the brief I treated as the deliverable in its own right, so it is specified in full.

4.1 Design principle

The final task is to forecast two months beyond the end of the labelled data. The validation scheme must therefore answer "*how well does a model trained on the past predict a contiguous future block it has never seen?*" — not "*how well does it interpolate among rows it was trained beside?*" Random k-fold would leak the future into the past and would flatter any model that memorises route-level rate levels.

I use a **blocked chronological holdout whose length exactly matches the real forecast horizon (61 days)**, plus a second, independent chronological split *inside* the training data for every decision that would otherwise contaminate the holdout.

4.2 The three-level split

Level	Purpose	Date range	Rows
Inner training	Fit proxies during feature screening	2025-01-01 to 2025-07-01	28,941
Inner validation	Choose the feature list and CatBoost iteration count	2025-07-02 to 2025-08-31	9,536
Outer training	Fit all final candidates	2025-01-01 to 2025-08-31	38,477
Outer holdout	Compare and select the final model	2025-09-01 to 2025-10-31	9,523
Final validation	Submitted predictions (labels never seen)	2025-11-01 to 2025-12-31	12,000

The pipeline asserts at runtime that the inner validation window ends strictly before the outer holdout begins, so no feature-selection decision can have touched a row used for final model comparison.

4.3 What each level is and is not allowed to do

- **The inner split decides the feature list and CatBoost's iteration count.** These are the decisions most prone to silent overfitting, so they are made entirely inside the training partition using fixed proxy configurations.
- **The outer holdout is scored once per candidate.** Every family, feature list, hyperparameter set and iteration count is frozen before the holdout is touched. No candidate is tuned against the holdout and re-scored.
- **All preprocessing is fitted on the training partition only.** Category encoders, frequency maps, coordinate maps and reference dates are learned inside each fit and applied outward; unseen categories map to a reserved value rather than to a holdout-derived statistic.

Because the outer holdout is both the selection surface and the reporting surface, \$129.51 is a **chronological model-selection estimate, not an unbiased test score**. With three eligible candidates the selection bias is small, but it is not zero and I do not present the number as a test result.

4.4 Selection rule, declared before scoring

Select the eligible candidate with the lowest finite MAE on the common chronological holdout. Exact-equality ties are broken deterministically by candidate name.

MAE is the primary metric for the same reason it is the training loss: with 1.4% multiplicative contamination, RMSE would rank models on how well they chase 144 rows.

4.5 A post-selection stability check

To confirm the holdout is not a lucky window, I refit the selected configuration on three expanding-window folds and scored each subsequent month:

Holdout month	Training rows	Holdout rows	MAE	RMSE	R ²
2025-08	33,718	4,759	\$157.29	\$627.81	0.8188
2025-09	38,477	4,670	\$128.98	\$623.58	0.8324
2025-10	43,147	4,853	\$125.27	\$654.49	0.8167

August is the worst month because it sits at the end of the sharpest level move of the year (\$8); the trend of improving MAE with more recent training data is itself evidence for the drift diagnosis.

5. Feature engineering

5.1 Feature groups evaluated

Rather than hand-pick features, I defined seven coherent groups and let a family-neutral screen choose between them:

Group	Contents
<code>basic_supplied</code>	Supplied operational fields plus missing/problem flags (8)
<code>basic_plus_date</code>	+ 20 calendar features: cyclic month/weekday/annual terms, holidays, month boundaries
<code>basic_plus_route</code>	+ endpoint categories, route/equipment frequencies, rare- and unseen-route indicators
<code>basic_plus_geographic</code>	+ coordinates, great-circle distance, distance-vs-great-circle difference and ratio, bearing, midpoint, distance band
<code>basic_plus_interactions</code>	+ log/quadratic distance and weight, weight-per-mile, distance × market, market × quote
<code>best_combined</code>	Union of all of the above (67 features)
<code>compact_calendar</code>	Minimal operational + compact calendar set

Route strings are deliberately *not* fed to HistGradientBoosting as raw categories — with 4,014 routes they exceed its 255-category limit — so route information enters through endpoints and training-only frequency counts instead.

Deliberately excluded: historical target encodings. Route-level mean or median historical rates are the obvious high-leverage feature here, and I left them out. With 537 routes seen fewer than five times, the encoding would be noise on the sparse tail, and computing it correctly across a chronological boundary requires expanding-window statistics that are easy to leak and hard to audit under time pressure. The frequency and rarity features capture the "how well do we know this lane" signal without touching the target.

5.2 Screening results

Each group was fitted with a fixed HGB proxy and a fixed CatBoost proxy on the inner split, and ranked by the unweighted mean of the two families' MAE — so no group could win merely by suiting one algorithm.

Feature set	Features	HGB MAE	CatBoost MAE	Mean (selection score)
<code>basic_plus_geographic</code>	24	\$134.89	\$119.71	\$127.30
<code>basic_plus_route</code>	23	\$134.82	\$124.56	\$129.69
<code>basic_supplied</code>	8	\$139.34	\$127.98	\$133.66
<code>basic_plus_interactions</code>	16	\$139.03	\$129.26	\$134.15
<code>best_combined</code>	67	\$134.63	\$149.07	\$141.85
<code>basic_plus_date</code>	28	\$137.56	\$153.46	\$145.51
<code>compact_calendar</code>	16	\$167.76	\$147.85	\$157.80

`basic_plus_geographic` wins, improving \$6.36 on the mean score over supplied fields alone. The same screen fixed CatBoost's iteration count at 165 via early stopping on distance-weighted rate-per-mile MAE — a metric proportional to posted-rate MAE, so the stopping criterion and the selection criterion agree.

Two results are worth flagging as honest negatives. `best_combined` is worse than its own components: 67 features dilute the signal, particularly for CatBoost. And `basic_plus_date` ranks second-to-last, which is why the selected model carries no calendar signal at all — a decision I revisit critically in §8.

5.3 Guardrails

`load_id`, `posted_rate` and `predicted_rate` are asserted absent from every candidate's feature matrix at runtime. `load_id` is used only for joins, ordering and integrity checks.

6. Modelling

6.1 Target transform and loss

Every model predicts **rate per mile**, and the pipeline multiplies by `distance` to recover a dollar rate. Distance explains most of the variance in `posted_rate` ($r = 0.909$) and rate-per-mile falls monotonically with haul length (\$2.75/mi under 250 miles to \$1.87/mi beyond 2,500). Modelling the ratio hands the model the strongest structural relationship for free and lets the trees spend their capacity on the residual pricing surface.

All models train with **absolute-error loss** and predictions are floored at \$1, guaranteeing the strictly-positive output the scorer requires.

6.2 Candidate comparison on the chronological holdout

Candidate	Family	Feature set	MAE	RMSE	R ²	sMAPE	Eligible	Selected
<code>hgb_compact</code>	HistGradientBoosting	<code>basic_plus_geographic</code>	\$129.51	\$640.32	0.8239	4.53%	Yes	Yes
<code>hgb_smoother</code>	HistGradientBoosting	<code>basic_plus_geographic</code>	\$130.26	\$640.40	0.8239	4.59%	Yes	No
<code>catboost_rate_per_mile</code>	CatBoost	<code>basic_plus_geographic</code>	\$141.07	\$643.10	0.8224	5.22%	Yes	No
<code>equipment_median_rpm</code>	Baseline	distance + equipment	\$229.10	\$670.66	0.8069	9.88%	No	No
<code>global_median</code>	Baseline	target only	\$1,148.92	\$1,569.42	-0.058	50.70%	No	No

Both baselines are computed on the identical holdout rows so the improvement claim is like-for-like. Note that the equipment-median baseline already reaches $R^2 = 0.807$ — with distance in the denominator, R^2 is a weak discriminator on this problem, which is precisely why MAE is the selection metric.

CatBoost led the inner screen (\$119.71 vs \$134.89) but lost the outer comparison. That reversal is expected rather than alarming: its iteration count was fitted to the inner window and then frozen, so it enters the outer round with a stopping point tuned for a different two-month regime. Reporting the reversal rather than quietly dropping CatBoost is the point of running both families.

6.3 Selected model

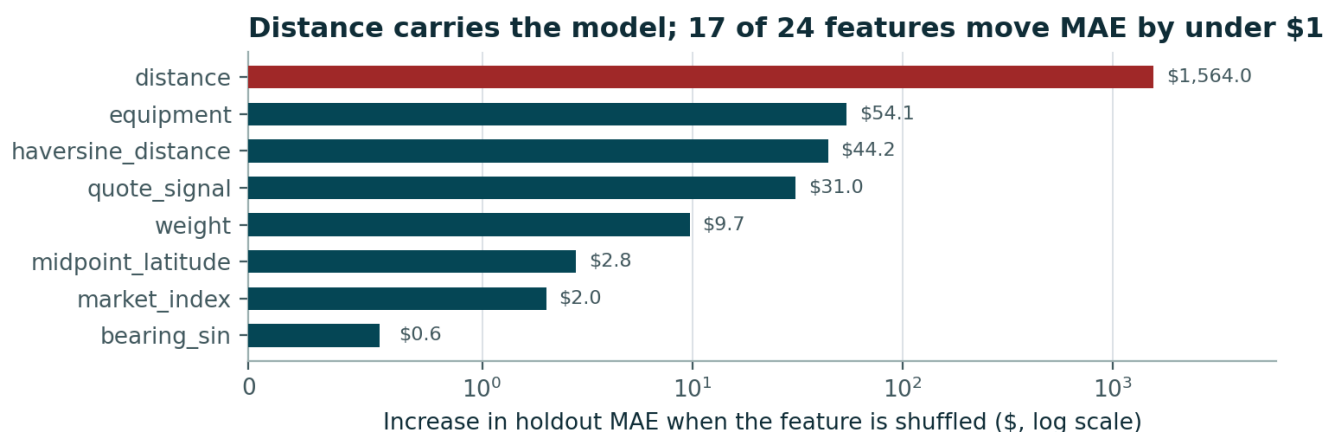
```
HistGradientBoostingRegressor (candidate: hgb_compact)
feature set : basic_plus_geographic (24 features, 2 categorical)
loss       : absolute_error on rate-per-mile
max_iter   : 250          learning_rate : 0.06
max_leaf_nodes : 15      min_samples_leaf : 30
l2_regularization : 3.0  random_state : 42  early_stopping : off
```

Metric	Chronological holdout
MAE	\$129.51
RMSE	\$640.32
R ²	0.8240
sMAPE	4.53%

Categorical features use an ordinal encoder fitted inside each training fit; unseen categories map to a reserved value, which is what makes the 8 unseen validation cities survivable.

The selected model is refit on **all 48,000 labelled rows** before generating the submission, so the shipped model sees two months more data than the model that was scored.

6.4 What the model actually uses



Permutation importance on the holdout (4,000-row sample, MAE increase when each feature is shuffled) is stark: distance is worth \$1,564, equipment \$54, great-circle distance \$44, `quote_signal` \$31, `weight` \$10. **17 of 24 features move MAE by less than \$1**, and six have marginally negative importance.

`market_index` scores only \$2.02 — but this measures sensitivity to *within-window* shuffling, and `market_index` barely varies inside the Sep-Oct holdout. It does not license the conclusion that the model is insensitive to a market-wide level shift, and \$8 shows it is not.

The practical read: the geographic block earned its place through great-circle distance and a handful of coordinate terms, and roughly half the feature list could be dropped with no measurable cost. A leaner model would be cheaper to serve and easier to monitor.

7. Error analysis

7.1 Where the error lives

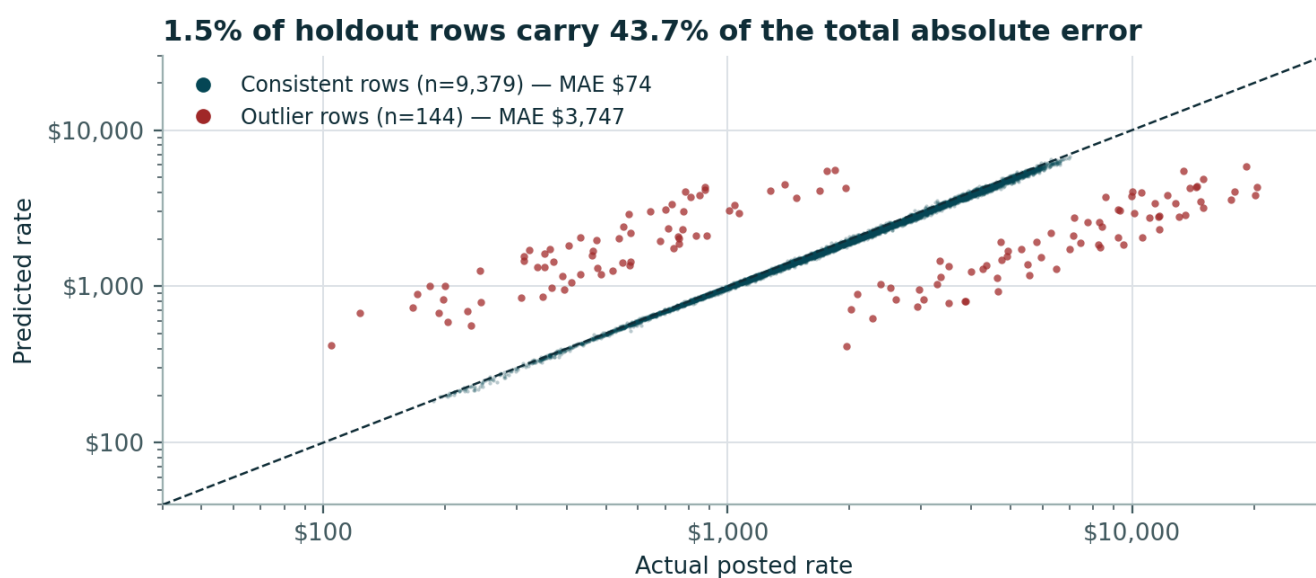
Dimension	Segment	Rows	MAE	Mean error
Distance	Short (<250 mi)	550	\$14.86	-\$7
	Medium (250–800)	3,356	\$56.58	-\$41
	Long (800–1,600)	3,157	\$147.09	-\$123
	Very long (>1,600)	2,460	\$232.08	-\$190
Equipment	Dry Van	5,360	\$121.46	-\$101
	Flatbed	1,770	\$131.71	-\$98
	Reefer	2,393	\$145.92	-\$118
Target band	Low (bottom 25%)	2,381	\$49.89	+\$22
	Medium	4,761	\$78.37	-\$68
	High (top 25%)	2,381	\$311.40	-\$305
Route familiarity	Common (≥ 10 loads)	5,900	\$134.57	-\$108
	Rare (<10)	3,602	\$121.59	-\$100
	Unseen	21	\$66.97	-\$67

Absolute error scales with haul length, as it must — a 4.5% error on a \$5,000 load is \$225. Relative error is flat to slightly improving with distance (sMAPE 2.6% short, 4.7% very long).

The important line is **route familiarity: unseen routes are not harder**. Rare and unseen lanes score as well as common ones, which confirms the decision to price from distance, geography and equipment rather than from lane memory. It is also the strongest available evidence that the 8 unseen validation cities are a manageable risk — though the holdout contains no unseen *cities* at all, so this remains an inference, not a measurement.

The **High** target band's MAE of \$311 is the contamination showing through: that band is where the inflated outliers land.

7.2 The 1.5% that dominates



Splitting the holdout by the \$3.3 classification:

Rows	Count	MAE	RMSE	Share of total absolute error
Consistent	9,379	\$73.98	\$103.90	56.3%
Contaminated	144	\$3,746.70	\$5,139.22	43.7%
All	9,523	\$129.51	\$640.32	100%

On the 98.5% of loads that follow the pricing surface, the model achieves \$74 MAE, **84.1% of predictions within $\pm 5\%$** and **98.4% within $\pm 10\%$** of the actual rate. The reported \$129.51 is that fit plus an irreducible tax from rows that are unpredictable by construction. Both numbers are true; only stating the first would be misleading, and only stating the second understates the model.

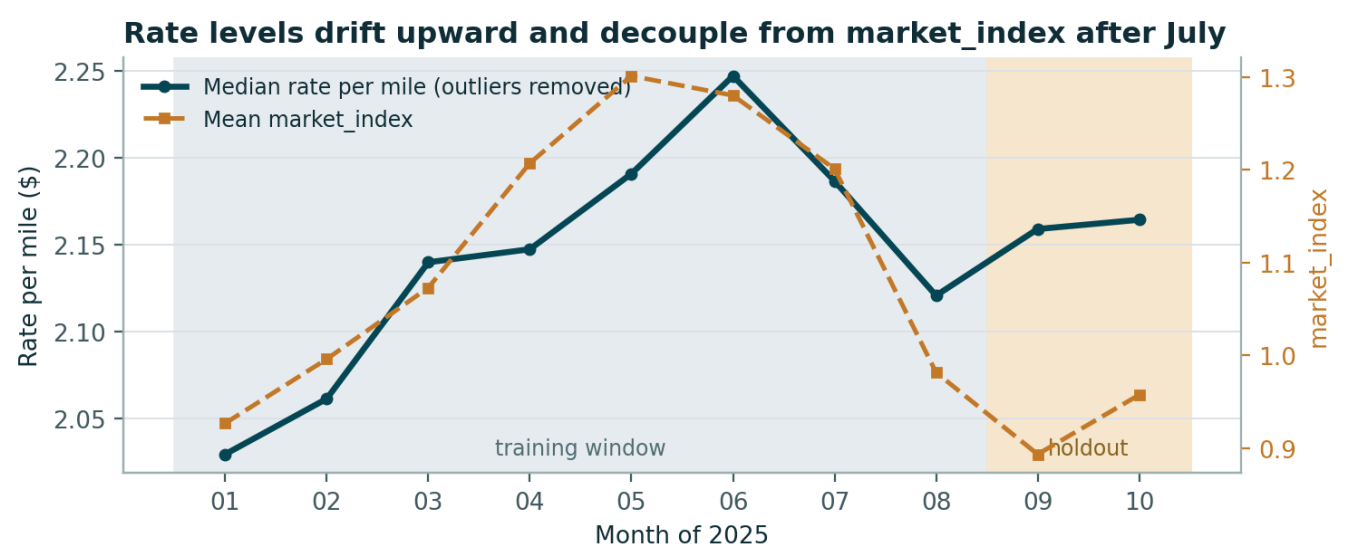
This also explains the RMSE-to-MAE ratio of 4.9x and the modest R^2 — 144 rows set both.

8. Known weakness: level drift

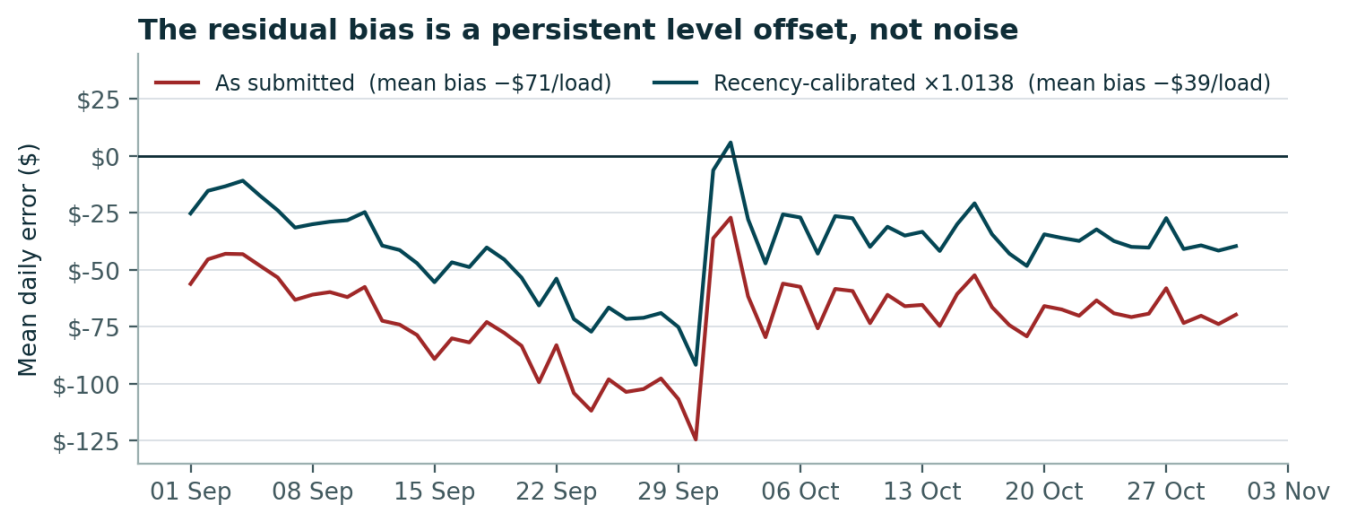
I would rather surface this than let a reviewer find it.

Every segment in §7.1 has a negative mean error. The model under-predicts by \$105 per load overall, and by \$71 on uncontaminated rows alone — roughly 3% of the mean rate. A uniform sign across all segments is a level offset, not scatter.

The cause is visible in the data:



Median clean rate-per-mile rose from \$2.03 in January to \$2.25 in June, then settled at \$2.16 in September–October. `market_index` tracked it until July and then fell away — dropping to 0.89 while rates held. A model with no calendar features and a weakly weighted `market_index` therefore learns an average level across a ten-month window and applies it to a period whose true level is higher.



The daily residual confirms it: a persistent offset with a within-month sawtooth (the target carries a ~2% day-of-month gradient and a ~1% weekday cycle that the selected feature set cannot represent), not random fluctuation.

8.1 Three corrections, measured

All measured on the same chronological holdout, with the selected model unchanged unless stated:

Variant	Holdout MAE	vs selected
Selected model as submitted	\$129.51	—
Trained on the last 120 days only	\$112.23	–13.3%
Add <code>days_since_start</code> to the feature list	\$109.21	–15.7%
Multiply predictions by a recency factor ($\times 1.0138$)	\$106.64	–17.7%

The recency factor is the median ratio of actual to predicted rate over the final 28 days of the **training** partition — no holdout row is used to compute it. Applying it after adding `days_since_start` yields a factor of 0.9999 and no further gain, confirming that both interventions address the same deficiency rather than stacking.

Because these variants were evaluated after the holdout had been used for selection, I validated the calibration separately by strict walk-forward — fit on a period, estimate the factor on a *held-out* month, test on everything after:

Fit through	Factor estimated on	Tested on	Rows	MAE before	MAE after	Improvement
2025-07-01	July	Aug–Oct	14,282	\$142.89	\$138.93	+2.8%
2025-08-01	August	Sep–Oct	9,523	\$122.68	\$107.59	+12.3%
2025-09-01	September	October	4,853	\$130.02	\$105.83	+18.6%

The correction helps in every fold and never hurts.

8.2 Why the screening missed it

This is a design flaw in my own feature screen, and it is instructive. Feature sets were evaluated as **fixed bundles**. `basic_plus_date` bundles a monotone time index together with 19 other calendar features — year, quarter, month, holiday flags — several of which cannot extrapolate past the training range and add pure noise. The bundle ranked second-to-last, and it took the one genuinely valuable feature down with it. Screening at bundle granularity is fast and defensible under a time constraint, but it cannot recover a single high-value feature buried in a weak group.

8.3 Effect on the submission

The submitted predictions average \$2,324.81 across 12,000 loads (median \$2,003.92, range \$171.71 – \$6,736.37), an implied median of \$2.126/mile against a recent actual level of ~\$2.16/mile. A recency factor computed from the last 28 days of the full development set is **1.0108**, implying the submission understates by roughly **\$25 per load, ~\$301,000 in aggregate**.

I left the submitted file uncalibrated: the declared selection rule chose the model on holdout MAE, and bolting on a post-hoc multiplier after seeing the holdout would break the discipline this report argues for. Documenting it, quantifying it and validating it out-of-sample is the honest alternative — and in production the calibration would be inside the pipeline, refreshed on a rolling window, rather than applied once by hand.

9. The fixed December scenario

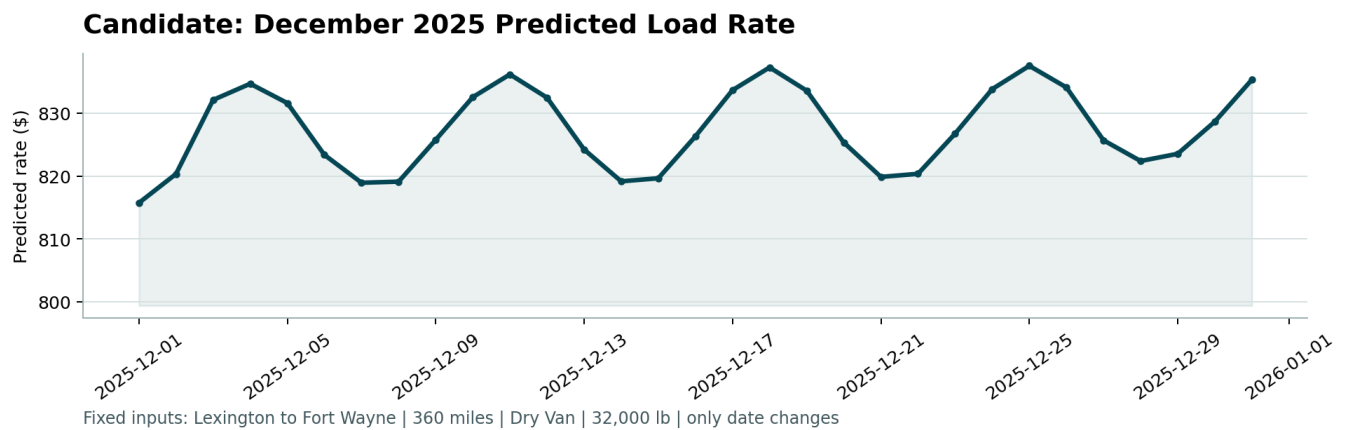
The scorer requires a 31-day series for a single fixed load: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb, with only the date changing.

That input carries a **reduced schema** — no `market_index`, no `quote_signal`, no coordinates. Rather than fabricate those values or feed the main model imputed medians, I ran a separate selection over four December-compatible candidates on the same chronological holdout, under the same rule:

Candidate	Family	Feature set	Features	MAE
<code>december_hgb_december_basic</code>	HistGradientBoosting	<code>december_basic</code>	16	\$108.78
<code>december_hgb_december_enriched_geo</code>	HistGradientBoosting	<code>december_enriched_geo</code>	32	\$110.24
<code>december_catboost_december_enriched_geo</code>	CatBoost	<code>december_enriched_geo</code>	32	\$125.70
<code>december_catboost_december_basic</code>	CatBoost	<code>december_basic</code>	16	\$126.04

The winner uses distance, weight, equipment, endpoints, log-distance and a compact calendar block — every field genuinely available in the December input, and nothing else. Its holdout MAE of \$108.78 beats the main model's \$129.51, which is consistent with \$8: the December feature set *does* contain calendar terms, so it captures part of the level drift the main model misses.

This chart is not output from the main validation model and should not be read as such. It is a separately validated reduced-schema scenario model, selected on the same holdout by the same rule.



The series spans \$815.74 – \$837.51 (mean \$827.41), a 2.7% range. The visible oscillation is a weekly cycle: mid-week loads price ~1% above Sunday loads, matching the day-of-week structure measured in the labelled data. The series is otherwise flat by construction — December sits beyond the training range, and gradient-boosted trees cannot extrapolate a trend, so the time index saturates at its last observed value. A production forecast for a horizon this far out would need an explicit trend model or a market-forward input; within the constraints of this brief, a flat-with-weekly-seasonality series is the defensible answer rather than an extrapolated one.

10. Reproducibility and verification

The pipeline is deterministic by construction: seed 42, `PYTHONHASHSEED` set before interpreter start, every numerical backend pinned to a single thread (thread-reduction order changes split decisions), stable `date`, `load_id` mergesort ordering, sorted candidate iteration and name-based tie-breaking.

It also verifies itself at runtime:

- SHA-256 checksums of the supplied `score.py` and the pristine December template, checked before and after execution — a modified scorer aborts the run.
- A content-addressed run identity hashed over all source files and inputs, stored in the generated artifacts rather than hardcoded in documentation.
- Assertions that the candidate pool matches the documented set exactly, that the saved model's class matches its recorded family, and that no forbidden column reached a feature matrix.
- Cross-checks that `model_comparison.csv`, `december_model_comparison.csv` and `validation_metrics.json` agree on the selected model, family, feature set and MAE to exact float representation — the machine-readable artifacts cannot silently disagree with each other.
- Schema validation of both output files, re-read from disk after writing, so what is validated is exactly what a reviewer receives.

Independent verification. The committed artifacts were produced on Windows. I re-ran the selected configuration on Linux with a different dependency stack (pandas 3.0.2 vs 2.2.3, matplotlib 3.10.8, scikit-learn 1.8.0), refit on all 48,000 rows, and compared against the committed submission file:

Check	Result
<code>hgb_compact</code> holdout MAE	129.5120 — matches to 4 decimals
<code>hgb_smoother</code> holdout MAE	130.2605 — matches to 4 decimals
Maximum absolute difference across all 12,000 submitted predictions	9.09×10^{-13}

Cross-platform, cross-version, bit-level agreement on the shipped artifact.

11. Limitations

Stated plainly, in rough order of how much they would change a decision:

1. **The reported MAE is a model-selection estimate, not a test score.** The outer holdout was used both to select and to report. With three candidates the bias is small; it is not zero.
2. **Level drift is uncorrected in the submitted file** (§8). This is the largest known, quantified gap, worth roughly 16–18% of MAE and ~\$301k of aggregate understatement.
3. **Cold-start city performance is inferred, not measured.** Validation contains 8 unseen cities; the holdout contains none. The unseen-route evidence is encouraging but is not the same test.
4. **The feature list rests on one inner window and two fixed proxy configurations.** A different inner split could plausibly choose `basic_plus_route`, whose selection score was \$2.39 behind.
5. **Contaminated rows are unpredictable by construction.** No feature engineering will close the 43.7% of error they carry; only a different problem framing (e.g. predicting an interval, or flagging low-confidence loads for human review) would help.
6. **Hyperparameters were lightly explored.** Two HGB configurations and one CatBoost configuration; no systematic search, by choice, given the time constraint and the risk of overfitting a single holdout.
7. **No external data.** No market indices, fuel prices, seasonality tables, credentials or private APIs were used.

12. What I would do next

In priority order, with the expected payoff:

1. **Put the recency calibration inside the pipeline** — a rolling factor refreshed on the trailing 28 days, validated by walk-forward. Measured at 12–19% MAE improvement across three folds; roughly a day of work.
2. **Screen features individually, not only in bundles.** Adding `days_since_start` alone was worth 15.7%; the bundle containing it ranked second-to-last. A cheap forward-selection pass over the union of groups would have caught this.
3. **Ship an uncertainty band, not just a point estimate.** With 1.4% of loads contaminated at 0.2×–5×, a broker's real question is "how confident is this quote?". Quantile regression at the 10th/90th percentiles costs two extra fits and turns the outlier problem from a metric penalty into a product feature.
4. **Prune the feature list.** 17 of 24 features move MAE by under \$1; a leaner model is cheaper to serve and easier to monitor.
5. **Add drift monitoring on the served model.** The failure mode here is not a broken model, it is a slowly stale one. Tracking the median actual-to-predicted ratio weekly would surface it before it costs anything.
6. **Revisit route-level target encoding properly**, with expanding-window statistics and a smoothing prior for sparse lanes — the one high-leverage feature family I deliberately deferred.

Appendix A — Repository and how to run

<code>run_pipeline.py</code>	end-to-end reproduction, audits through scorer execution
<code>score.py</code>	supplied scorer, unmodified (checksum-verified)
<code>src/features.py</code>	leakage-safe cleaning and feature construction
<code>src/train.py</code>	model wrappers, candidates, iteration learning
<code>src/evaluate.py</code>	metrics, error analysis, output and artifact validation
<code>src/reporting.py</code>	EDA artifacts and figures
<code>artifacts/</code>	machine-readable outputs (see Appendix B)
<code>validation_predictions.csv</code>	12,000 submitted predictions
<code>december-chart-inputs.csv</code>	31 completed December rows
<code>scorer_results/</code>	scorer-generated December chart

```
python -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
PYTHONHASHSEED=42 python run_pipeline.py
```

The pipeline locates and validates inputs, audits data quality and distribution shift, builds the chronological splits, screens features, compares candidates, refits both selected models on all labelled rows, generates and schema-validates both outputs, runs the unmodified scorer, and cross-checks every generated artifact for consistency. `README.md` and this report are static and are checksum-verified as unchanged by the run.

Appendix B — Artifact index

Artifact	Contents
<code>artifacts/validation_metrics.json</code>	Canonical selections, metrics, splits, versions, run identity
<code>artifacts/model_comparison.csv</code>	Main candidate comparison including both baselines
<code>artifacts/december_model_comparison.csv</code>	Reduced-schema December comparison
<code>artifacts/feature_ablation.csv</code>	Family-neutral feature screening results
<code>artifacts/feature_importance.csv</code>	Permutation importance for the selected model
<code>artifacts/error_analysis.csv</code>	Segment-level holdout error
<code>artifacts/holdout_predictions.csv</code>	Row-level holdout actuals, predictions and residuals
<code>artifacts/temporal_stability.csv</code>	Expanding-window monthly diagnostics
<code>artifacts/data_audit.json</code>	Schema, quality, shift and target-distribution audit
<code>artifacts/input_manifest.json</code>	Input checksums and run fingerprint
<code>artifacts/scorer_output.txt</code>	Captured output of the supplied scorer